Test Case ID	Target Feature / Operation	Outcome Before Refactoring	Outcome After Refactoring	Engineering Performance Improvement / Observation
TC-01	Basic Element Insertion	**PASS** — Items were inserted using global array state.	**PASS** — Items inserted through DynamicArray::add().	Data is now encapsulated inside a class instead of relying on mutable global variables.
TC-02	Dynamic Memory Resize	**FAIL** — Array resized from capacity 5 to 10, but the old heap allocation was leaked.	**PASS** — Array resizes correctly and old memory is released using delete[].	Prevents cumulative heap leaks during repeated resizing while maintaining amortized $O(1)$ insertion.
TC-03	Array Traversal / Display	**FAIL** — printAll() used i <= currentCount, causing an out-of-bounds read.	**PASS** — Traversal uses i < size_.	Eliminates invalid memory reads and prevents garbage output or possible crashes.
TC-04	Search for Existing Element	**PASS** — findItem(30) correctly returned index	**PASS** — DynamicArray::find(30) returns index	Same $O(n)$ worst-case complexity, but redundant inner-loop

		2.	2.	overhead was removed.
TC-05	Search for Missing Element	**PASS** — Search for 999 returned -1.	**PASS** — Search for 999 returns -1.	Maintains correct linear-search behavior with simpler implementation .
TC-06	Valid Element Deletion	**PASS** — Valid deletion shifted elements correctly.	**PASS** — removeAt() performs a single-pass shift.	Removes unnecessary nested-loop logic while retaining $O(n)$ worst-case deletion.
TC-07	Negative Index Validation	**PASS** — Negative index was rejected.	**PASS** — Invalid index is rejected through centralized validation.	Validation is now handled consistently by the data-structure class.
TC-08	Upper-Bound Index Validation	**FAIL** — remove_item_at(99) could access memory outside the valid array range.	**PASS** — removeAt(99) throws std::out_of_range safely.	Eliminates a critical memory-safety vulnerability and prevents undefined behavior.

TC-09	Matrix Transposition	**PASS** — Hardcoded 3 × 3 matrix was transposed correctly.	**PASS** — Matrix is represented using std::array and transposed correctly.	Matrix functionality is preserved while improving type safety and organization.
TC-10	Memory Cleanup / Program Exit	**FAIL** — dataArray was never released at program termination.	**PASS** — DynamicArray destructor automatically executes delete[].	RAII provides automatic resource cleanup and eliminates the explicit cleanup omission.
TC-11	Empty Array Operation	**NOT TESTED / UNSAFE** — Legacy code did not explicitly define safe empty-array behavior.	**PASS** — Empty array can be displayed safely and invalid deletion is rejected.	Adds explicit empty-state handling and prevents invalid memory access.
TC-12	Code Structure / Encapsulation	**FAIL** — Global variables and independent functions controlled shared state.	**PASS** — DynamicArray encapsulates data and operations.	Improves modularity, maintainability, testability, and reusability.